

41052 : Advanced Algorithms

Assignment 1: Algorithm Implementation Project

Written Report

University of Technology Sydney

Acesha Bello – 24741403

What I Built

The Algorithm

The Knuth-Plass line-breaking algorithm is a dynamic programming algorithm that is used to break up a paragraph into lines at once, rather than filling each line greedily. The algorithm does this in this way to minimise “badness”. Badness is defined in the algorithm as a way to minimise the white space of any given line of a paragraph. It chooses the set of breaks that minimises the total **demerits** of the paragraph, so that no line is much looser or tighter than those surrounding it.

In the algorithm, text is broken into 3 types: **boxes** (unbreakable, such as words), **glue** (spaces with an ideal width and how far they can stretch and shrink) and **penalties** (places a break is allowed, encouraged, or forbidden). A break can occur at a glue that comes before a box, or at a penalty below infinity. For any line in a paragraph, the adjustment ratio **r** says how much every space must stretch (positive **r**) or shrink (negative **r**) for the line to stretch and reach its target width. A line with a ratio below -1 is not possible, and a ratio above the tolerance is rejected. Badness is $\min(100 \times |r|^3, 1000)$, and the demerits of a line are $(\text{line_penalty} + \text{badness})^2$, which is then adjusted by the penalty at the break [1]. Finding the optimal breaks is then a shortest-path problem over all the breakpoints and solved by dynamic programming.

Implementation and Key Design Decisions

I implemented the algorithm in C++17 as a header, *knuth_plass.hpp* and built a command line tool *kpcli* that uses the algorithm as a core component. In this, breaking is a dynamic program over the multiple breakpoints. Each possible way of reaching a breakpoint produces a *State* that records its total demerits, its fitness class and line number and an integer index **prev** that points to the state it came from. The optimal solution is recovered in **break_lines** by following **prev** pointers back from the cheapest state at the final compulsory break and then reversing the list.

States are stored in a **std::vector<State>** and refer to each other by index rather than by pointer, so the references stay valid when the vector reallocates as it grows. Line widths are computed in $O(1)$ from three prefix-sum arrays (width, stretch, shrink) instead of rescanning the items for every new line.

Rather than keeping every candidate at a breakpoint, **run_pass** keeps a list of active states that can still start a future line. A state is deactivated when the line starting from it comes too long (**r < -1**), since lines only get longer as the break position moves forward, or when a forced break passes it.

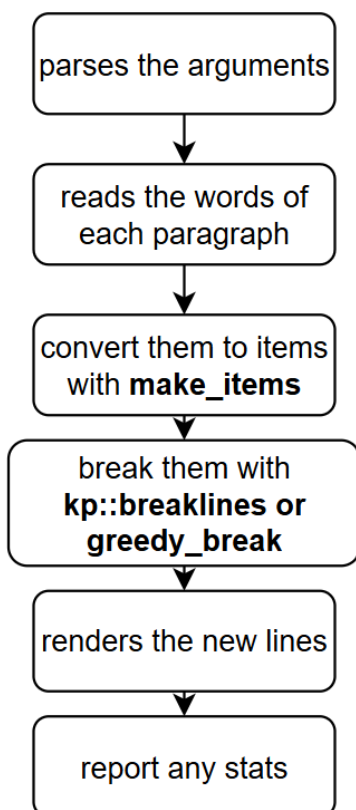
At each breakpoint, **run_pass** keeps only the cheapest candidate per (fitness class, width class) pair. The width class supports a different target width per line through **Params::line_widths**. The final width is repeated for any following lines after that. The CLI I implemented only uses a single width, but the actual implementation of the algorithm supports varying widths.

The algorithm also includes fitness demerits (adjacent lines whose fitness classes differ by more than one), flagged demerits (consecutive breaks at flagged penalties) and an emergency second pass (unlimited tolerance, allowing overfull lines) used when the first pass finds no solution.

There were a few simplifications made relative to the textbook version in my implementation. I did not implement hyphenation, if a word needed to be split between two lines. My CLI only generates boxes and glues. It does not generate flagged penalties, or penalties that print a hyphen. The glue between words in my CLI is **glue(1,1,0)** and has fixed width of 1, stretch of 1 and shrink of 0. Due to this, spaces cannot shrink and a line can not get smaller. I decided to measure the width of inputs in bytes, so one column per character.

Track B: The CLI Tool

The *kpcli* reads text from a file or standard input and treats any blank lines as paragraph separators. The tool breaks each paragraph independently and writes the result to standard output. Below is a flowchart of how the tool works:



The options in the tool are:

```
const char* USAGE =
    "how to use: ./kpcli [options] [INPUTFILE] \n"
    "\n"
    "options:\n"
    "  -w N, --width N      line width (default is 70)\n"
    "  -a MODE, --align MODE left | right | center | justify \n"
    "  --greedy             use greedy line breaking \n"
    "  --stats              printing statistics to the stderr\n"
    "  -h, --help           showing the help!\n";
```

How the algorithm sits inside it

Each paragraph passes through five stages in `kpcli.cpp`:

1. **Read** (`format_stream`). The input is read line by line and split on whitespace and a blank line ends a paragraph.
2. **Model** (`make_items`). All words become boxes, gaps between words become `glue(1, 1, 0)`, and `kp::end_paragraph()` appends the standard ending: a forbidden break, a "fill" glue with effectively infinite stretch, and a forced break. The fill glue lets the last line be short without being penalised.
3. **Break** (`format_paragraph`). This calls `kp::break_lines()`, or `greedy_break()` if `--greedy` was selected as an option. Both of these return `std::vector<kp::Line>`, so the later stages do not need to know which algorithm was selected to run.
4. **Render** (`render_line`). Each line range is turned back into words, and the alignment mode decides where the spare columns go. For justification, extra spaces are distributed as evenly as possible, with the remainder going to the later gaps. The last line of any paragraph will not be justified.
5. **Output and statistics** (`format_paragraph`, `format_stream`). Each line is printed; with `--stats`, per-line width, slack, adjustment ratio (only the Knuth-Plass algorithm) and overfull status are written to stderr. This is then followed by the paragraph's total demerits (and whether the emergency pass was needed) and a final summary of the stats including how many lines used.

Interface Decisions

I chose to have my output go to **stdout** for text and any statistics or diagnostics to **stderr**. This allowed for one of my CLI options (`--stats`) to be used while still formatting the text. The input to the CLI can be a file or standard input. If an exit code 2 is raised, it means there is a usage error and an incorrect option has been selected. An exit code of 0 means the algorithm and CLI ran successfully.

Another option I chose to have was the greedy comparison. It takes in the same items and inputs, and returns the same Line type. The main difference here is how breaking is decided and where, but the stats populated and how the text is rendered remains the same. This allows for the two outputs to be compared directly.

The summary line labelled “**sum of slack**” is the sum of squared slack over any lines that are not the last line of a paragraph or overfull. Squaring this value was a choice as I noticed it penalises one very loose line more than several slightly loose ones, and this keeps in line for what the Knuth-Plass algorithm is trying to optimise for.

Worked Example

Input: input.txt, a simple paragraph

Option: width of 70

```
PS C:\Users\aeesha.bello\source\repos\adv_algo_assignment1\build\release> ./kpcli -w 70 input.txt
I remember that the Gabilan Mountains to the east of the valley
were light gay mountains full of sun and loveliness and a kind of
invitation, so that you wanted to climb into their warm foothills
almost as you want to climb into the lap of a beloved mother. They
were beckoning mountains with a brown grass love. The Santa Lucias
stood up against the sky to the west and kept the valley from the
open sea, and they were dark and broodingÖÇounfriendly and dangerous.
I always found in myself a dread of west and a love of east. Where
I ever got such an idea I cannot say, unless it could be that the
morning came over the peaks of the Gabilans and the night drifted back
from the ridges of the Santa Lucias. It may be that the birth and
death of the day had some part in my feeling about the two ranges of
mountains.
```

Input: input.txt, a simple paragraph

Option: width of 70, --greedy comparison

```
PS C:\Users\aeesha.bello\source\repos\adv_algo_assignment1\build\release> ./kpcli -w 70 --greedy input.txt
I remember that the Gabilan Mountains to the east of the valley were
light gay mountains full of sun and loveliness and a kind of
invitation, so that you wanted to climb into their warm foothills
almost as you want to climb into the lap of a beloved mother. They
were beckoning mountains with a brown grass love. The Santa Lucias
stood up against the sky to the west and kept the valley from the open
sea, and they were dark and broodingÖÇounfriendly and dangerous. I
always found in myself a dread of west and a love of east. Where I
ever got such an idea I cannot say, unless it could be that the
morning came over the peaks of the Gabilans and the night drifted back
from the ridges of the Santa Lucias. It may be that the birth and
death of the day had some part in my feeling about the two ranges of
mountains.
```

Knuth-Plass

I remember that the Gabilan Mountains to the east of the valley were light gay mountains full of sun and loveliness and a kind of invitation, so that you wanted to climb into their warm foothills almost as you want to climb into the lap of a beloved mother. They were beckoning mountains with a brown grass love. The Santa Lucias stood up against the sky to the west and kept the valley from the open sea, and they were dark and brooding—friendly and dangerous. I always found in myself a dread of west and a love of east. Where I ever got such an idea I cannot say, unless it could be that the morning came over the peaks of the Gabilans and the night drifted back from the ridges of the Santa Lucias. It may be that the birth and death of the day had some part in my feeling about the two ranges of mountains.

Greedy

I remember that the Gabilan Mountains to the east of the valley were light gay mountains full of sun and loveliness and a kind of invitation, so that you wanted to climb into their warm foothills almost as you want to climb into the lap of a beloved mother. They were beckoning mountains with a brown grass love. The Santa Lucias stood up against the sky to the west and kept the valley from the open sea, and they were dark and brooding—friendly and dangerous. I always found in myself a dread of west and a love of east. Where I ever got such an idea I cannot say, unless it could be that the morning came over the peaks of the Gabilans and the night drifted back from the ridges of the Santa Lucias. It may be that the birth and death of the day had some part in my feeling about the two ranges of mountains.

Both of these tests produce 13 lines. Greedy packs the first 3 lines almost full, which leaves it with shorter words to carry into the following lines. Knuth-Plass gives up on the fullness of some lines early (breaking after “valley”) rather than breaking after “were”. This leaves it so that there is no line that is shorter than 63 columns. The sum of squared slack falls from 271 in greedy to 227 in Knuth-Plass, and the worst line improves from a slack of 10 to 7. Lines 9-13 are the exact same in both, because once the two outputs reach the same break the remaining choices will always coincide.

Example

Input: input.txt, a simple paragraph

Option: width of 70, output stats of each line.

```
PS C:\Users\aesha.bello\source\repos\adv_algo_assignment1\build\release> ./kpcli -w 70 --stats input.txt
paragraph 1:
I remember that the Gabilan Mountains to the east of the valley
  line 1: 63/70  ratio 0.636364  slack 7
were light gay mountains full of sun and loveliness and a kind of
  line 2: 65/70  ratio 0.416667  slack 5
invitation, so that you wanted to climb into their warm foothills
  line 3: 65/70  ratio 0.5  slack 5
almost as you want to climb into the lap of a beloved mother. They
  line 4: 66/70  ratio 0.307692  slack 4
were beckoning mountains with a brown grass love. The Santa Lucias
  line 5: 66/70  ratio 0.4  slack 4
stood up against the sky to the west and kept the valley from the
  line 6: 65/70  ratio 0.384615  slack 5
open sea, and they were dark and broodingÖCounfriendly and dangerous.
  line 7: 69/70  ratio 0.111111  slack 1
I always found in myself a dread of west and a love of east. Where
  line 8: 66/70  ratio 0.285714  slack 4
I ever got such an idea I cannot say, unless it could be that the
  line 9: 65/70  ratio 0.357143  slack 5
morning came over the peaks of the Gabilans and the night drifted back
  line 10: 70/70  ratio 0  slack 0
from the ridges of the Santa Lucias. It may be that the birth and
  line 11: 65/70  ratio 0.384615  slack 5
death of the day had some part in my feeling about the two ranges of
  line 12: 68/70  ratio 0.142857  slack 2
mountains.
  line 13: 10/70  ratio 6e-08  slack 60
demerits: 3783.41
total: 1 paragraph(s), 13 line(s)
sum of slack: 227
```

```
PS C:\Users\aesha.bello\source\repos\adv_algo_assignment1\build\release> ./kpcli -w 70 --greedy --stats input.txt
paragraph 1:
I remember that the Gabilan Mountains to the east of the valley were
  line 1: 68/70  slack 2
light gay mountains full of sun and loveliness and a kind of
  line 2: 60/70  slack 10
invitation, so that you wanted to climb into their warm foothills
  line 3: 65/70  slack 5
almost as you want to climb into the lap of a beloved mother. They
  line 4: 66/70  slack 4
were beckoning mountains with a brown grass love. The Santa Lucias
  line 5: 66/70  slack 4
stood up against the sky to the west and kept the valley from the open
  line 6: 70/70  slack 0
sea, and they were dark and broodingÖCounfriendly and dangerous. I
  line 7: 66/70  slack 4
always found in myself a dread of west and a love of east. Where I
  line 8: 66/70  slack 4
ever got such an idea I cannot say, unless it could be that the
  line 9: 63/70  slack 7
morning came over the peaks of the Gabilans and the night drifted back
  line 10: 70/70  slack 0
from the ridges of the Santa Lucias. It may be that the birth and
  line 11: 65/70  slack 5
death of the day had some part in my feeling about the two ranges of
  line 12: 68/70  slack 2
mountains.
  line 13: 10/70  slack 60
total: 1 paragraph(s), 13 line(s)
sum of slack: 271
```

What I Learned

The emergency pass runs on almost every paragraph that I tested (barring the sample shown above). It occurs especially when the column widths start to go below 60. When I ran `--stats`, the demerits line said (**emergency pass**), on my test. In my testing, I did realise this only occurred based on how I defined my **glue**. Each space has stretch 1 and shrink 0, and the default tolerance is 1.0. A 40-column line that is 8 columns short with 6 spaces needs a ratio of 1.33, which exceeds the tolerance, so the first pass through of the algorithm will reject it. As there is no shrink available, the first pass often finds no complete solution and every paragraph falls back to the emergency pass with unlimited tolerance. The output still looked reasonable, but it highlighted to me that the tolerance is highly relative to how much stretch a glue (or space) will provide.

My early research described the algorithm as checking every earlier feasible breakpoint A for each breakpoint B[2]. My implementation instead keeps an active list and drops any states that can no longer start a possible line, which avoids considering breaks that can never work.

As I was building and testing the relevant options of my CLI, I realised my width option was parsed but never stored. I had set the default width to 70, and when I tested “**—width 40**”, the line widths never changed. The option was being parsed and validated but I never assigned the result to any value so the default was always used. In debugging the **parse_argument** method, I noticed after checking the argument input, it would just continue on. I had to add this assignment [**options.width = static_cast<int>(width)**] so it would convert the width to an int and then store the resulting integer as an option for the width to run on.

I initially took the ‘`std::map<std::pair<int, std::size_t>, State> best`’ pruning on trust (see AI section). As I debugged my code, I was able to understand why this was valid. Every candidate that is created at the same breakpoint **b**, shares the same break item and have the same flagged status. This means they all start the next line at ‘`line_start_after(b)`’. The only things that affect a candidate's future cost are its fitness class (through fitness demerits) and its width class (through the next line's target width). If two candidates agree on both, the one with more accumulated demerits needs to be discarded as it cannot lose the optimum or lead to a better final solution.

AI Use

I used Claude for most of the implementation of the algorithm and ChatGPT for reviewing the final CLI implementation that was built. I also used ChatGPT for planning milestones to achieve in the CLI implementation, and interpreting errors I received when running the CLI in Powershell.

I originally used Claude to implement the entire algorithm. However the implementation it provided me was incredibly advanced and I struggled to understand parts of the code and how it related to the algorithm. The code outputted was more production-style, and went out of the scope of the key parts of the algorithm I meant to explore. It went into UTF-8 handling, hyphenation, multiple input files, emergency handling and also variable line widths (see Git Commit #c71613c on September 10). Asking it to explain the code, did not help me understand how it related and built up to the whole algorithm. I simplified the scope, scaffolded the implementation based on my own research, existing implementations of the algorithm in other languages and what key sections I required to exist. This included item types, line widths, rendering and alignment. Using this, I was able to expand on the implementation myself, and build up to the full complete solution.

When I started testing my implementation with my CLI, all the outputs were blank. The active list I was building of breakpoints, was being replaced at every breakpoint (**active = new_active**). This was discarding all earlier nodes that had been touched. At the first space, a line containing one word was far too loose to be accepted, so no new state was created. This was because the starting node had already been discarded, the active list was empty and no solution could be found. I fixed this by keeping earlier nodes active and removing a node only when its line has become too long ($r < -1$) or when a forced break passes it.

The main part of the implementation that I struggled with and took on trust was trying to find the best: (**std::map<std::pair<int, std::size_t>, State> best**). I understood this pruning as trying to keep only the cheapest state for each combination of fitness class and width class. The assumption made here is that, once two states have reached the same breakpoint with the same fitness class and width class, their future possibilities and future costs are determined by the same relevant state information. Therefore, the state with the higher accumulated demerits cannot then produce a better solution than the cheaper state, and can safely be discarded.

Video Walkthrough

<https://youtu.be/mFtdFgwFbzA>

References

[1] Defoe Project, "**Knuth-Plass Line-Breaking Algorithm — Revisited.**" Available at: <https://defoe.sourceforge.net/folio/knuth-plass.html>

[2] alex-panda, "**KnuthPlassLinebreak.**" GitHub repository. Available at: <https://github.com/alex-panda/KnuthPlassLinebreak>

[Creating a new CLI Automation Tool for C++ Builds \(using c++ to build c++ tools that build c++ ^-^\)](#)

[It is never too late to write your own C/C++ command-line utilities – Daniel Lemire's blog](#)

[Create a C++ UI – Build a Command-Line Interface \[7/24\] – Gautier Talks About Technology](#)